

MECHA CONNECT

Master Handbook

AI-Powered Roadside Assistance & Vehicle Services Platform

Version — 1.0.0 (RC1 · Frontend Lock Candidate 1.9.2)

Build — Flutter 3.29.2 · Dart 3.7.2

Authors — Mecha Connect Engineering Team

Date — 2026-08-05

Copyright — © 2026 Mecha Connect. All rights reserved.

Classification — Internal / Confidential

Table of Contents

Chapter 1 — Executive Summary	6
Chapter 2 — Abstract	6
Chapter 3 — Vision, Mission and Goals	7
3.1 Vision	7
3.2 Mission	7
3.3 Goals	7
3.4 Engineering Values	8
Chapter 4 — Problem Statement	8
4.1 The Problem	8
4.2 Existing System	8
4.3 Limitations of the Existing System	9
4.4 Persona Pain Points	9
4.5 The Need for Mecha Connect	9
Chapter 5 — Solution and Product Overview	10
5.1 Product Overview	10
5.2 Features	11
5.3 Benefits	12
5.4 Target Users	12
Chapter 6 — Business Model	13
6.1 Revenue Streams	13
6.2 Cost Structure and Unit Economics	13
6.3 SWOT	13
6.4 Competitor Analysis and Market Opportunity	14
6.5 Growth and Future Business	14
Chapter 7 — Product Requirements Document	14
7.1 Functional Requirements	14
7.2 Non-Functional Requirements	15
7.3 Use Cases	16
7.4 User Stories	16
7.5 Acceptance Criteria (global)	16
Chapter 8 — System Architecture	17
8.1 Flutter Architecture	17

8.2 Source Tree (frozen)	17
8.3 Provider Architecture	17
8.4 Repository Pattern (the frozen seam)	18
8.5 Service Layer	18
8.6 Dependency Graph	18
8.7 Navigation Graph (summary)	19
Chapter 9 — Technology Stack	19
9.1 Delivered (RC1)	19
9.2 Planned (Sprint 2)	20
9.3 Toolchain	20
Chapter 10 — Module Documentation	20
10.1 Splash, Onboarding and Authentication	20
10.2 Home (features/home)	20
10.3 Mechanic (features/mechanic)	21
10.4 Fuel Delivery (features/fuel_delivery)	21
10.5 Marketplace (features/marketplace)	21
10.6 AI Assistant (features/ai)	21
10.7 Profile (features/profile)	21
10.8 Orders (bottom_bar/order_screen.dart)	21
10.9 Settings	22
10.10 Vehicle Location (features/vehicle_location)	22
Chapter 11 — User Workflows	22
11.1 Authentication Workflow	22
11.2 Mechanic Booking Workflow	22
11.3 Fuel Delivery Workflow	22
11.4 Marketplace Workflow	22
11.5 AI Workflow	23
11.6 Profile Workflow	23
11.7 Complete Journey	23
Chapter 12 — Navigation Documentation	23
12.1 Style and Rules (frozen)	23
12.2 Root Flow	23
12.3 Bottom Navigation Shell (5 tabs, IndexedStack)	24
12.4 Stacked Flows	24

12.5 Back Stack and Cross-Module Edges	24
12.6 Transition Language (frozen)	24
12.7 Deep Links (future)	25
Chapter 13 — UI Design System	25
13.1 Colors (lib/theme/app_colors.dart)	25
13.2 Typography	25
13.3 Spacing (AppSpacing)	25
13.4 Cards, Dialogs and Bottom Sheets	25
13.5 Buttons and Animations	25
13.6 Dark Mode	26
13.7 Accessibility	26
13.8 Responsive Design	26
Chapter 14 — Database Blueprint	26
14.1 Conventions	26
14.2 Entity–Relationship Overview	26
14.3 Core Tables	26
14.4 Indexes and Constraints	27
14.5 Future Schema	27
Chapter 15 — API Contract	28
15.1 General Conventions	28
15.2 Endpoints (module → repository methods)	29
15.3 Request / Response Example — AI Diagnosis	29
15.4 Validation and Errors	30
15.5 Authentication	30
Chapter 16 — Testing	30
16.1 Strategy	30
16.2 Inventory (162 tests)	31
16.3 Gates	31
16.4 Runtime QA, Accessibility, Performance	32
Chapter 17 — Sprint History	33
17.1 Lessons Learned	34
17.2 Architecture Evolution	34
Chapter 18 — Deployment Roadmap	34
18.1 Sprint 2 (Backend Integration)	34

18.2 Sprint 3 (Production Polish)	34
18.3 CI/CD	34
18.4 Monitoring, Logging and Security	35
Chapter 19 — Future Scope	35
Chapter 20 — Known Limitations and Risk Register	35
20.1 Known Limitations (accepted at RC1)	35
20.2 Risk Register	36
Chapter 21 — Appendix	37
21.1 Glossary	37
21.2 Packages	37
21.3 Licenses	37
21.4 Credits	37
21.5 References	37
21.6 Version History	38

AI-Powered Roadside Assistance & Vehicle Services Platform

Version — 1.0.0 (RC1 · Frontend Lock Candidate 1.9.2) **Build** — Flutter 3.29.2 · Dart 3.7.2 **Authors** — Mecha Connect Engineering Team **Date** — 2026-08-05 **Copyright** — © 2026 Mecha Connect. All rights reserved. **Classification** — Internal / Confidential. This document is the official technical reference for the Mecha Connect platform. Do not distribute outside the project without authorization.

Chapter 1 — Executive Summary

Mecha Connect is an AI-powered roadside assistance and vehicle services platform. It connects vehicle owners, mechanics, fuel delivery partners, spare part sellers, and administrators through one unified mobile application. A single app provides mechanic booking, on-demand fuel delivery, a spare parts marketplace, AI-assisted vehicle diagnostics, real-time live tracking, and a wallet with rewards — described internally as "Uber + Swiggy + AI Assistant" for vehicle services.

At RC1, the product is a **feature-complete Flutter frontend**, frozen as a **Frontend Lock Candidate**. Every module is delivered and verified:

Gate	Result
Static analysis (<code>flutter analyze</code>)	No issues found! (0 errors, 0 warnings)
Automated tests (<code>flutter test</code>)	162 / 162 passing
Modules	Home, Services (Mechanic / Fuel / Marketplace), Orders, AI, Profile, Vehicle Location
Runtime & navigation audits	PASS — 0 P0/P1/P2 defects
Accessibility & responsive audits	PASS — all safe fixes applied
Certification documentation	9 canonical documents + this handbook

The entire data layer runs on **mock repositories** that faithfully simulate production latency, failure, and retry behavior, so the UI already behaves exactly as it will against the real backend. Sprint 2 replaces the repository internals with a FastAPI + PostgreSQL backend behind the frozen contract; the UI and its 162 tests do not change.

This handbook is the official, continuous technical reference for the platform: product vision, requirements, architecture, module documentation, navigation, design system, data and API contracts, testing, history, deployment roadmap, and known limitations. It is written as a single book rather than a bundle of documents, so an engineer, reviewer, investor, or faculty evaluator can read it end to end.

Chapter 2 — Abstract

Vehicle breakdowns are a high-stress, time-critical problem, especially in India where reliable roadside help is discovered mostly offline through word of mouth. The market is fragmented: mechanics, fuel delivery, and spare part sales operate as disconnected services with opaque pricing, and no unified platform exists for a stranded user.

Mecha Connect addresses this with a single application that closes the loop from symptom to resolution: an AI assistant diagnoses the fault and estimates cost, a verified mechanic is booked and tracked live, fuel can be delivered to a parked vehicle, and parts can be ordered — all paid through a wallet and documented in one order history.

This handbook documents the RC1 release: the complete Flutter frontend, its feature-first architecture, frozen design and navigation, the Sprint 2 API and database contracts, and the verification evidence (162/162 tests, clean static analysis). It is the single source of truth for developers onboarding onto the codebase and for the Sprint 2 backend integration team.

Chapter 3 — Vision, Mission and Goals

3.1 Vision

Mecha Connect is an AI-powered roadside assistance ecosystem connecting vehicle owners, mechanics, fuel partners, spare part sellers, and administrators through a unified platform. The long-term vision is a self-healing mobility support layer: whenever a vehicle needs help — on the road, in the driveway, or before a long journey — Mecha Connect is the first and fastest way to get it resolved.

3.2 Mission

Build the most **reliable, fast, and trustworthy** roadside assistance and vehicle-services platform. Every screen exists to help a user in distress: parts delivered, a mechanic booked, fuel at the door, a diagnosis before the shop opens. Because the product is used in emergencies, the engineering bar is higher than for a typical consumer app — failure paths are first-class features, and transparency (honest pricing, verified professionals, honest "coming soon" copy) is a core value.

3.3 Goals

Product and business goals (per the Product Requirements Document):

Goal	Target
Mechanic response	ETA under 15 minutes
Booking completion rate	Above 80%
User rating	Above 4.0 out of 5
Crash-free sessions	Above 99.5%
Page load time	Under 2 seconds
Code quality	flutter analyze 0 issues; full suite green on every commit
Build success	100%

3.4 Engineering Values

Value	Practice
Reliability	Every mock repository supports deterministic failure injection; loading, empty, and error states are rendered and tested.
Speed	No rebuild-every-tick listeners; <code>context.select</code> for fine-grained rebuilds; lazy lists everywhere.
Trust	Transparent pricing breakdowns, honest copy, no dead buttons.
Accessibility	Semantics on all interactive controls; 44–48dp touch targets; dark-mode parity.
Maintainability	<code>flutter analyze</code> at 0 issues and 162/162 tests as the permanent bar.

Chapter 4 — Problem Statement

4.1 The Problem

Vehicle breakdowns are stressful, and owners struggle to find trusted help quickly. The specific pain points are:

- **No unified platform** — no single service exists for roadside assistance in India; users juggle phone numbers, offline contacts, and multiple apps.
- **Opaque pricing** — users consistently overpay for emergency repairs because quotes are not transparent.
- **Offline discovery** — mechanic discovery is word-of-mouth; there is no trusted, rated directory.
- **Fragmented fuel delivery** — getting fuel to a stranded or parked vehicle is ad hoc and unreliable.
- **Disconnected parts** — spare part sourcing is separate from repair services, causing long, opaque wait times.

4.2 Existing System

The incumbent landscape (competitive analysis against UrbanPiper, GoMechanic, and Park+) covers fragments of the need:

Capability	UrbanPiper	GoMechanic	Park+	Mecha Connect
AI diagnosis	✗	✗	✗	✓

Capability	UrbanPiper	GoMechanic	Park+	Mecha Connect
Mechanic booking	✓	✓	✗	✓
Real-time tracking	✗	✗	✗	✓
SOS emergency	✗	✗	✗	✓
Fuel delivery	✗	✗	✗	Planned
Spare parts marketplace	✗	✗	✗	Planned
Partner app	✓	✗	✗	Planned

4.3 Limitations of the Existing System

Each incumbent lacks one or more of the capabilities the market needs most: none of them combine AI diagnosis, live tracking, and SOS with the full multi-service set. Users therefore fall back to fragmented tools: a directory app for the mechanic, a separate app for fuel, cash and ambiguity for payment, and no structured way to track the service arriving.

4.4 Persona Pain Points

Persona	Scenario	Pain
Rajesh (28, Hyderabad, Honda Activa)	Bike breaks down twice a year	No way to know which mechanic to trust; needs a verified mechanic within 10 minutes.
Priya (32, Bangalore, Hyundai i10)	Stranded at night	Worried about safety; needs a tracked mechanic arrival, SOS, and verified profiles.
Vikram (40, Chennai, 10 delivery bikes)	Frequent breakdowns, inconsistent costs	Needs bulk service booking, cost tracking, and reliable mechanic ratings.

4.5 The Need for Mecha Connect

Vehicle owners need a single trusted platform that diagnoses a problem, arranges help, tracks its arrival, and handles payment — with transparent pricing and verified professionals. Mecha Connect is that platform, closing the loop between the AI diagnosis, the mechanic, fuel, parts, and payment.

Chapter 5 — Solution and Product Overview

5.1 Product Overview

Mecha Connect is one application that covers the full roadside-assistance journey: mechanic booking, fuel delivery, a spare parts marketplace, AI diagnostics, wallet, and rewards — with live tracking and SOS safety.

5.2 Features

Feature	Description	Priority	Delivered
Splash	Animated brand entry (glow, scale, rotation), 60 FPS	P0	✓
Onboarding	3-slide introduction to the app	P0	✓
Authentication	Email/password login, sign-up, forgot password; on-device session	P0	✓
Home dashboard	Greeting, vehicle health, quick services, marketplace teaser, offers	P0	✓
Mechanic booking	9-step flow: list → details → service → summary → confirm → track → complete → review	P0	✓
Vehicle service request	AI-driven vehicle form with diagnosis	P1	✓
Fuel delivery	Station selection, quantity, price estimate, live tracking, invoice	P1	✓
Spare parts marketplace	Catalog, categories, cart, checkout, wishlist, coupons, orders	P2	✓
AI assistant	Chat, structured diagnosis, history, cross-module actions	P1	✓
Profile	Profile, vehicles, addresses, wallet, rewards, settings, support	P1	✓
Orders	Unified order feed across all modules	P1	✓
Dark mode	System/light/dark with dedicated palette	P0	✓

Feature	Description	Priority	Delivered
Responsive layout	Mobile/tablet/desktop via AppResponsive	P0	✓

5.3 Benefits

- **AI diagnosis before booking** — describe symptoms and receive a fault prediction with a cost estimate before committing.
- **End-to-end booking** — from diagnosis to payment in one seamless flow.
- **One ecosystem** — mechanic, fuel, and parts in a single app.
- **Real-time tracking** — know exactly when help arrives and who is coming.
- **Verified professionals** — rated mechanics with transparent pricing.
- **Transparency** — explicit cost lines, honest availability, no dead buttons.

5.4 Target Users

Segment	Detail
Primary	Vehicle owners (two-wheeler, three-wheeler, car) in semi-urban and urban India; daily commuters aged 18–45; tech-savvy app users.
Secondary	Fleet operators (delivery, logistics); women drivers (safety-focused); long-distance travelers.
Tertiary	Mechanics (service providers), fuel station partners, spare part sellers.

Chapter 6 — Business Model

6.1 Revenue Streams

Stream	Model	Margin	Timeline
Mechanic booking	15–20% commission per transaction	■60–90 per booking	Sprint 2
Fuel delivery	Delivery fee + fuel margin	■20–50 per delivery	Sprint 1.7+
Marketplace listings	Monthly seller subscription	■99 per seller / month	Sprint 1.8+
Featured mechanics	Priority placement fee	■499 per mechanic / month	Sprint 2
AI diagnosis API	B2B subscription	■999 per fleet / month	Sprint 3
In-app ads	Sponsored listings	Variable	Sprint 3

6.2 Cost Structure and Unit Economics

Monthly MVP cost: cloud (Firebase + FastAPI) ■15,000; AI API (Gemini) ■5,000; maps (OpenStreetMap / tile server) ■2,000; development and marketing as project budget permits.

Unit economics: customer acquisition cost (CAC) ■150; average order value (AOV) ■450; gross margin per booking ■67–90; customer lifetime value (LTV) ■2,250 (5 × ■450); payback after approximately 3 transactions.

6.3 SWOT

	Positive	Negative
Internal	Strengths — complete frozen frontend; AI diagnosis differentiator; single-platform coverage (mechanic + fuel + parts); verified professionals; live tracking.	Weaknesses — no live backend yet; AI diagnosis is mock at RC1; brand unknown; iOS not configured.
External	Opportunities — large unorganized Indian roadside-services market; fleet/B2B AI API; 5-city expansion; partner ecosystem (mechanic, vendor portals).	Threats — incumbents expanding coverage; Gemini rate limits/cost; store policy risk; GPS/network dependence on low-end devices.

6.4 Competitor Analysis and Market Opportunity

Only Mecha Connect combines AI diagnosis, mechanic booking, live tracking, and SOS in one product; fuel delivery and the spare parts marketplace are the differentiators being prepared for Sprint 2. The Indian roadside assistance market is large and unorganized, giving a single-platform entrant a clear positioning: the one app that handles a breakdown end to end.

6.5 Growth and Future Business

Growth path: **Seed** (Sprint 1–2, one city) → **Series A** (Sprint 3–4, five cities) → **Growth** (Sprint 5+, partner app + B2B fleet). Acquisition channels: Google Ads ("mechanic near me"), Instagram/Facebook targeting, a referral program (■50 credit per referral), and partnerships with parking lots and service stations.

Projected monetization: Sprint 1 ■0 → Sprint 2 ■15,000 → Sprint 3 ■50,000 → Sprint 4 ■1,00,000 → Sprint 5 ■5,00,000 per month. Future revenue extends via the B2B AI diagnosis API, sponsored listings, a partner/vendor portal, and subscription plans for frequent users and corporate fleets.

Chapter 7 — Product Requirements Document

7.1 Functional Requirements

Per-feature functional requirements are specified with explicit screen states: every module screen defines **loading** (skeleton), **error** (retry button with message), **empty** ("no results" illustration), **denied location** (permission dialog), **GPS disabled** (settings redirect), and **ready** (full UI) states.

Module	Functional requirement
Splash	Branded animated entry; routes by login/onboarding state; no bypass.
Onboarding	3 slides; marks onboarding complete; only shown to new users.
Auth	Email + password login, sign-up, forgot password; validation; session persisted on-device.
Home	Dashboard with quick services, nearby services, marketplace teaser, activities, offers; search.
Mechanic	List, details, service selection, booking summary, confirmation, live tracking, completion, rating.
Fuel	Station selection by location, quantity, price estimate, payment, live tracking, invoice, history.
Marketplace	Catalog, categories, brands, offers, coupons, cart, checkout, wishlist, search, orders.
AI	Chat, structured diagnosis with cost estimate, history, pin/rename/delete, cross-module actions.

Module	Functional requirement
Profile	Profile edit, vehicles (default/promote/CRUD), addresses (default/CRUD), wallet, rewards, orders, settings.
Orders	Unified feed of parts/mechanic/fuel/AI entries with category filters.
Settings	Notification settings, privacy & security, theme picker, about, support.

7.2 Non-Functional Requirements

Category	Requirement
Performance	Page load under 2 seconds; ETA under 15 minutes; no rebuild-every-tick listeners; lazy lists.
Reliability	Crash-free sessions above 99.5%; deterministic failure injection; retry paths exercised.
Usability	Booking completion above 80%; user rating above 4.0; onboarding friction minimized.
Accessibility	Semantics on interactive controls; touch targets 44–48dp; reduced-motion support; dark-mode parity.
Responsiveness	Correct layout at 320–768dp+ in light and dark (tested widths 320/360/390/412/600/768).
Code quality	<code>flutter analyze</code> 0 issues; <code>flutter test</code> 162/162; Conventional Commits; no TODO/FIXME/HACK.
Security	No secrets in code; no real credentials in mock paths; input validation on all forms.
Maintainability	Feature-first modules; repository seam as the only data access point.
Localization (future)	Hindi, Telugu, Tamil planned post-MVP.

7.3 Use Cases

ID	Actor	Use case	Priority
UC-1	Owner	Log in to the app (or register)	P0
UC-2	Owner	Get AI diagnosis and cost estimate for a fault	P1
UC-3	Owner	Book a verified mechanic and track arrival live	P0
UC-4	Owner	Order fuel delivery to a parked vehicle	P1
UC-5	Owner	Buy spare parts from the marketplace	P2
UC-6	Owner	Pay with wallet and view unified order history	P1
UC-7	Owner	Save vehicles and addresses; set defaults	P1
UC-8	Owner	Trigger SOS and view emergency actions	P2

7.4 User Stories

Story	Acceptance basis
As Rajesh, I want to find a verified mechanic near me so that I can be helped within 10 minutes.	Nearby mechanics list with ratings, distance, ETA, availability.
As Priya, I want to see the mechanic live on a map so that I know exactly when help arrives at night.	Live tracking screen with ETA and status transitions.
As Vikram, I want consistent cost breakdowns so that fleet servicing is predictable.	Booking summary with explicit line items and surcharge.
As an owner, I want a diagnosis before booking so that I do not overpay.	AI diagnosis returns causes, severity, and estimated cost.
As an owner, I want one place to see all my orders so that I never lose track.	Unified Orders tab across all modules.

7.5 Acceptance Criteria (global)

Each delivered story must satisfy:

- flutter analyze — 0 errors, 0 warnings.
- Widget/unit tests for the affected surface — passing.
- No crash on hot reload; no RenderFlex overflow at 360px and 600px+ widths.
- Dark mode renders correctly for every screen touched.
- Interactive controls have semantics/tooltips and adequate touch targets.
- All routes resolve to real screens (no dead buttons).

Chapter 8 — System Architecture

8.1 Flutter Architecture

The app is a **Flutter (Material 3)** single-page application built on a **feature-first** module layout, **Provider** for state management, and a **repository pattern** for data access. Every module owns its models, repository, provider, screens, widgets, services, and navigation helpers.

8.2 Source Tree (frozen)

```
lib/
■■■ main.dart                # entry, env load, root providers, SplashScreen
■■■ app_wiring.dart          # single source of truth for the provider graph
■■■ theme/                   # AppColors, AppSpacing/AppElevation, AppResponsive,
#                             # ThemeProvider, AppTheme (light/dark), app_theme_helpers
■■■ bottom_bar/
#   ■■■ bottom_navigation.dart # 5-tab GNav + IndexedStack shell
#   ■■■ order_screen.dart     # Orders tab (listens to OrderStore)
■■■ parts/order_data.dart    # shared order store + ordersList singleton
■■■ starting_screen/         # Splash → Onboarding/Login + HomeDashboard +
#                             # ServiceSelectionScreen
■■■ homescreen/drawerscreen.dart # ProfileDrawer
■■■ services/                # location_provider, geocoding_service, ...
■■■ widgets/                 # shared UI (order_card, location_header, app_loading, ...)
■■■ features/
    ■■■ auth/                # Login / SignUp / ForgotPassword
    ■■■ home/                # Home repository + models
    ■■■ ai/                  # chat, diagnosis, history
    ■■■ marketplace/         # catalog, cart, checkout, wishlist, orders
    ■■■ mechanic/           # booking, live tracking, ratings
    ■■■ fuel_delivery/       # booking, tracking, invoice/receipt
    ■■■ profile/             # profile, vehicles, addresses, wallet, rewards
    ■■■ vehicle_location/    # vehicle location flow
```

8.3 Provider Architecture

Providers are constructed in one place — `app_wiring.dart` — above `MaterialApp`, so every screen (pushed or tab) reads the same instance:

#	Provider	Constructed with
1	ThemeProvider	shared_preferences theme_mode

#	Provider	Constructed with
2	LocationProvider	injectable or default
3	AuthProvider	AuthService(AuthRepository())
4	HomeProvider	HomeRepository()
5	MechanicProvider	repository injectable
6	AiProvider	owns exactly ONE AiRepository (shared with services)
7	ProfileProvider	ProfileRepository(NotificationSettingsStore)
8	FuelProvider	needs LocationProvider
9	MarketplaceProvider	injectable

Non-provider singletons: `orderStore` (`OrderStore` extends `ChangeNotifier`) and the seeded `ordersList` in `lib/parts/order_data.dart`; `addMarketplaceOrder` calls `orderStore.notify()` so the Orders tab and Profile history stay in sync.

8.4 Repository Pattern (the frozen seam)

- Every module has a repository that is the **only** data access point.
- Screens never call HTTP and never import backend clients.
- Providers call repositories; widgets read providers.
- Mock repositories simulate latency (e.g. 700–800ms) and support deterministic failure injection (`failForFirstCalls(N)`), so retry/error UI is real.
- No real network calls exist anywhere in the mock paths (audit-verified).

8.5 Service Layer

Pure business logic lives in `services/`: price math (`FuelService`), field validation (`ProfileService/ValidationService`), location and geocoding (`LocationProvider`, `geocoding_service.dart`), and tracking simulation. Widgets never compute business values.

8.6 Dependency Graph

```

Screens/Widgets → Providers (ChangeNotifier) → Repositories → Mock data
                ■                                ■→ (Sprint 2: HTTP client)
                ■→ Services (pure logic) → Repositories

```

8.7 Navigation Graph (summary)

Splash routes by login/onboarding state to the 5-tab shell, Login, or Onboarding. The shell is an IndexedStack (Home, Services, Orders, AI, Profile). Pushed flows: mechanic, fuel, marketplace, AI, and profile screens all use imperative Navigator .push via module navigation.dart helpers. Full detail in Chapter 12.

Chapter 9 — Technology Stack

9.1 Delivered (RC1)

Layer	Technology	Version
Framework	Flutter (Material 3)	3.29.2
Language	Dart	3.7.2
State management	provider + nested	6.1.5 / 1.0.0
Bottom navigation	google_nav_bar	5.0.7
Layout preview	device_preview	1.2.0
Persistence	shared_preferences	2.5.3
Env config	flutter_dotenv	5.2.1
Geo math	latlong2	0.9.1
Geolocation	geolocator	13.0.4
Permissions	permission_handler	11.4.0
Maps	flutter_map	7.0.2
HTTP (geocoding; Sprint 2 client)	http	1.6.0
Lints (dev)	flutter_lints	5.0.0

9.2 Planned (Sprint 2)

Layer	Technology	Purpose
API server	FastAPI	Backend implementation of the frozen API contract
Database	PostgreSQL 15+	Persistence per the database blueprint
Cache	Redis	Session/cache layer
Auth	Firebase Auth + JWT	Real authentication
AI	Gemini API	Chat + diagnosis (RAG over the knowledge base)
Maps	OpenStreetMap tiles	Map rendering (replacing mock geometry)
Observability	Firebase Crashlytics / Analytics	Crash-free + metrics tracking
CI/CD	GitHub Actions	Analyze, test, build, distribute

9.3 Toolchain

Git + GitHub (hosting and workflow), VS Code (IDE), OpenCode (AI-assisted development), Python 3.13 (backend and ML tooling: `generate_data.py`, `train.py`, `build_rag_index.py` for the FAISS knowledge base).

Chapter 10 — Module Documentation

10.1 Splash, Onboarding and Authentication

- **Splash** (`lib/main.dart`): timed branded sequence with glow/scale/rotation

animation at 60 FPS; routes by `is_logged_in` and `onboarding_completed` via a 450ms fade `pushReplacement`.

- **Onboarding**: 3 slides, pill indicators, brand colors; marks onboarding

complete on finish.

- **Auth** (`features/auth`): `AuthRepository` → `AuthService` → `AuthProvider`;

Login / SignUp / ForgotPassword screens with email format and password ($\text{length} \geq 6$) validation; login state persisted under `is_logged_in`.

10.2 Home (features/home)

`HomeRepository.fetchHomeData()` (800ms) → `HomeData` (quick services, nearby services, marketplace items, activities, offers). `HomeProvider` + dashboard and search screens. Home teaser cards (marketplace / nearby / activity) are intentional static placeholders for Sprint 2 wiring.

10.3 Mechanic (features/mechanic)

Models: MechanicInfo, MechanicService, MechanicCategory, MechanicReview, booking/tracking records. Repository: 4 mechanics (m1–m4, one unavailable), 8 categories, 8 services, reviews r1–r8. MechanicProvider orchestrates list → details → service → vehicle form → summary → confirmation → live tracking → job completed → rating. The vehicle form uses the AI module's DiagnosisService (no real HTTP).

10.4 Fuel Delivery (features/fuel_delivery)

Models: FuelType, FuelVehicle, FuelStation, DeliveryLocation, FuelOrder, PriceEstimate, TrackingInfo, Invoice, FuelPartner, OrderStatus (requested → accepted → fuelPacked → partnerAssigned → enRoute → arrived → delivered / cancelled). FuelRepository (700ms): stations by location, lifecycle, INV-<orderId> invoices, seeded history FUEL-2026-0005..0009. FuelService.calculatePrice computes the full cost breakdown. Screens: home, booking, payment, confirmation, live tracking, complete, receipt, history.

10.5 Marketplace (features/marketplace)

Models: Product, Category, Brand, Offer, Coupon, Cart, MarketplaceOrder, Review. Frozen catalog: 40 products (one out-of-stock), 10 categories, 15 brands, 3 offers, 3 coupons. MarketplaceProvider handles catalog, cart, checkout, wishlist; placeOrder writes the Orders tab via addMarketplaceOrder. ProductCard uses context.read + context.select (wishlist-only rebuilds). Screens: home, category, product detail, cart, checkout, order success, search, wishlist.

10.6 AI Assistant (features/ai)

Models: Conversation, ChatMessage, AiResponse/AiBlock/AiActionButton, Diagnosis, QuickAction, SuggestedQuestion. AiRepository (900ms, one instance shared with AiService and DiagnosisService) seeds 5 conversations (2 pinned) and answers via a keyword knowledge base; diagnoseVehicle returns a structured payload (causes, severity, estimated cost, confidence, recommended service). AiProvider.loadHome/refreshHome use _mergeReloaded() to preserve user conversations. Screens: AI home, chat, diagnosis, conversation history, conversation detail. Cross-module actions: book mechanic, fuel, search parts.

10.7 Profile (features/profile)

Models: UserProfile (seeded Jagadeesh Gowda, Pro tier), ProfileVehicle (veh-101..102), SavedAddress (addr-101..102), WalletData (balance ■1,200, 2,450 pts), RewardsData, ProfileStats, NotificationSettings, EmergencyContact, Coupon. ProfileRepository (800ms, failure injection) supports vehicle/address CRUD with default-promotion, wallet, rewards, stats (12 services), and notification settings via NotificationSettingsStore. Screens: profile, edit, vehicles, vehicle detail, addresses, wallet, rewards, order history, notifications, privacy & security, support, about.

10.8 Orders (bottom_bar/order_screen.dart)

Tab 2 renders the shared ordersList grouped by category filter tabs (All / Parts / Mechanic / Fuel / AI). It stays mounted in the IndexedStack, rebuilds on OrderStore notifications, and "Explore Services" switches to the Services tab.

10.9 Settings

Notification settings (push/email/SMS/marketing toggles, persisted), privacy & security screen, theme picker (system/light/dark), about, and support — all under the Profile module.

10.10 Vehicle Location (`features/vehicle_location`)

Flow supporting vehicle location with permission handling (GPS denied/GPS disabled states with manual-address fallback), using `geolocator`, `permission_handler`, and `latlong2`; covered by 8 tests.

Chapter 11 — User Workflows

11.1 Authentication Workflow

```
Splash → is_logged_in?  
  ■■ yes → BottomNavigation (5-tab shell)  
  ■■ no + onboarding done → Login  
  ■    ■■ valid credentials → BottomNavigation  
  ■    ■■ forgot password → ForgotPassword → Login  
  ■    ■■ sign up → SignUp → BottomNavigation  
  ■■ no + no onboarding → Onboarding (3 slides) → Login  
Logout (Profile) → confirm → LoginScreen
```

11.2 Mechanic Booking Workflow

```
Service card "Breakdown / Garage" → VehicleFormPage  
  → MechanicHomeScreen → NearbyMechanicsScreen → MechanicDetailsScreen  
  → SelectServiceScreen → BookingSummaryScreen (cost breakdown)  
  → BookingConfirmationScreen (ETA)  
  → LiveTrackingScreen (status: connecting → searching → assigned →  
    en route → arrived → in progress → service completed)  
  → JobCompletedScreen → RatingReviewScreen (stars + review)
```

11.3 Fuel Delivery Workflow

```
FuelHomeScreen → pick station + quantity → FuelBookingScreen  
  → PaymentScreen → OrderConfirmationScreen  
  → LiveTrackingScreen (requested → accepted → fuelPacked → partnerAssigned →  
    enRoute → arrived → delivered)  
  → OrderCompleteScreen → ReceiptScreen (INV-<orderId>)
```

11.4 Marketplace Workflow

```
MarketplaceHomeScreen → CategoryScreen / ProductDetailScreen  
  → CartScreen → CheckoutScreen (address, payment, coupon)  
  → OrderSuccessScreen (writes Orders tab via orderStore)  
  → "View Orders" → Orders tab  
WishlistScreen → ProductDetailScreen (wishlist toggle)  
SearchScreen → ProductDetailScreen
```

11.5 AI Workflow

```
AiHomeScreen → suggested question → ChatScreen
  → "Run Guided Diagnosis" → DiagnosisScreen (structured result + actions)
  → conversation history → detail (rename/delete via bottom sheet)
Cross-module: diagnosis/quick actions → MechanicHome / FuelHome / MarketplaceHome
```

11.6 Profile Workflow

```
ProfileScreen → EditProfile / MyVehicles → VehicleDetail
  / SavedAddresses (add/edit sheets) / Wallet / Rewards
  / My Orders / NotificationSettings / PrivacySecurity / Support / About
  / Theme picker / Logout
```

11.7 Complete Journey

Sign up → onboarding → home dashboard → break down → AI diagnosis → book mechanic → live track → job completed → rate → pay via wallet → unified order in Orders tab → order history in Profile. Every step is reachable and every transition tested.

Chapter 12 — Navigation Documentation

12.1 Style and Rules (frozen)

- Imperative `Navigator.push(MaterialPageRoute(...))`; the only named route is `/ (Splash)`.
- Each feature has a `navigation.dart` with route constants and push helpers (`openWallet`, `openMyVehicles`, `openNotificationSettings`, `openSupport`, `aiFadeRoute`, `profileFadeRoute`, `marketplace` `openProduct/openCategory`).
- Back affordance: `AppBar` leading `IconButton(tooltip: 'Back')`; conditional back buttons use `Navigator.canPop() + maybePop()`.
- No dead routes: every route constant resolves to a real screen. Intentional "coming soon" behavior is limited to roadmap features not yet in the codebase (Battery, Towing, full activity history, nearby service details).

12.2 Root Flow

```
main() → MultiProvider(buildRootProviders) → MyApp → DevicePreview → MaterialApp
  initialRoute '/' → SplashScreen
    ■■ is_logged_in → BottomNavigation
    ■■ no + onboarding → Login
    ■■ no onboarding → Onboarding
```

12.3 Bottom Navigation Shell (5 tabs, IndexedStack)

Index	Tab	Screen
0	Home	HomeDashboard
1	Services	ServiceSelectionScreen
2	Orders	Orderscreen
3	AI	AiHomeScreen
4	Profile	ProfileScreen

All five tabs stay mounted; hidden tabs are offstage but findable and stateful. GNav bar with a 250ms built-in transition; body uses IndexedStack.

12.4 Stacked Flows

- **Mechanic:** `VehicleFormPage → MechanicHome → Nearby → Details → Select Service → Summary → Confirmation → LiveTracking → JobCompleted → Rating (push/pushReplacement` along the way).
- **Fuel:** `FuelHome → FuelBooking → Payment → Confirmation → LiveTracking → OrderComplete → Receipt`.
- **Marketplace:** `MarketplaceHome → Category/Product/Cart → Checkout → OrderSuccess`.
- **AI:** AiHome → Chat | Diagnosis | History → Detail.
- **Profile:** `Profile → Edit | Vehicles | Addresses | Wallet | Rewards | Orders | Notifications | Privacy | Support | About`.

12.5 Back Stack and Cross-Module Edges

From	To	Trigger
Home / Services	Mechanic form, Fuel, Marketplace	Service cards
AI chat/diagnosis	Mechanic, Fuel, Marketplace	AI action buttons
Marketplace checkout	Orders tab	addMarketplaceOrder + orderStore.notify()
Profile	Auth	Logout (pushAndRemoveUntil)
Orders tab	Services tab	"Explore Services"

12.6 Transition Language (frozen)

- Splash → entry: 450ms fade pushReplacement.

- AI module: `aiFadeRoute` (220ms fade). Profile module: `profileFadeRoute` (220ms / 180ms). Other modules: `MaterialPageRoute` (platform default).

12.7 Deep Links (future)

Sprint 2 plans app links / Firebase Dynamic Links so notifications can route to specific screens (order tracking, chat, diagnosis).

Chapter 13 — UI Design System

13.1 Colors (`lib/theme/app_colors.dart`)

Brand: `brandOrange` #F15A22, `brandOrangeLight` #FF7A4D, `brandOrangeDark` #D44A15, `brandOrangeSoft` #FFF3ED; `brandBlue` #4285F4 and shades. Greys `grey50` . . `grey900`. Dark palette: `darkBg` #0E1117, `darkCard` #1A1D24, `darkSurface` #232833, `darkBorder` #313846, `darkPrimary` #FF6A2A, `text` #FFFFFF / #B7BDC8 / #6B7280. Semantic: `success` #10B981, `error` #EF4444, `warning` #F59E0B, `info` #3B82F6 (each with light/dark pairs). Text: `textOnPrimary` #FFFFFF. Glass and gradient-stop tokens complete the set. **Rule:** never hardcode colors in widgets — always `context.*` tokens.

13.2 Typography

Display/headings favor **Space Grotesk** (e.g. dashboard greeting, w700); body uses the platform default stack. Sizes follow a 12/13/14/16/20/28/32 hierarchy with w500/w600/w700 weights; scaling via `AppResponsive.scaleFont` (clamped 0.88–1.2).

13.3 Spacing (`AppSpacing`)

4px base scale: `xxs` 2 · `xs` 4 · `sm` 8 · `md` 12 · `base` 16 · `lg` 20 · `xl` 24 · `xxl` 32 · `xxxl` 40 · `xxxxl` 48 · `hero` 64. Radii: 6/10/14/18/22/28/999. Never use raw padding/margins.

13.4 Cards, Dialogs and Bottom Sheets

Cards: `radiusMd-Lg`, soft shadows (`AppElevation` low–highest), standard padding `cardH/cardAll`. Dialogs: theme default with title/actions (confirmations, logout, theme picker). Bottom sheets: vehicle/address forms, conversation rename/delete, payment selection.

13.5 Buttons and Animations

Buttons: pill-shaped (`radiusFull`), brand gradient/orange primary, tonal secondary; disabled states dark-safe. Animations: splash stagger, card entrance, hero autoplay (disabled under reduced motion), skeleton/shimmer and AI typing indicators — all repeat loops gated behind `MediaQuery.disableAnimationsOf`.

13.6 Dark Mode

ThemeProvider (system/light/dark, persisted theme_mode); AppTheme.light / AppTheme.dark wired into MaterialApp. Widgets read brightness through the app_theme_helpers.dart context getters (context.cardBg, context.border, context.accent, context.textTertiary), so dark-mode safety is structural, not per-widget branching.

13.7 Accessibility

Icon-only buttons carry tooltip:/semanticLabel;; touch targets ≥ 48 dp (44dp on dense product cards); merged semantics with ExcludeSemantics for decorative groups (stars $\rightarrow$ "Rated X out of 5", carousels $\rightarrow$ "Promotion banner, offer N of M"); reduced-motion honored; dark-mode contrast verified.

13.8 Responsive Design

Breakpoints: mobile < 600 · tablet 600–1024 · desktop ≥ 1024 . AppResponsive.responsive<T> picks per-device values; gridColumns returns 2/3/4; horizontalPadding 16/24/32; ConstrainedContent caps content width (480 on tablet/desktop). Tested at 320/360/390/412/600/768dp in light + dark.

Chapter 14 — Database Blueprint

14.1 Conventions

PostgreSQL 15+; UUID primary keys; TIMESTAMPTZ timestamps; money as NUMERIC(12, 2) (INR); statuses as VARCHAR with CHECK constraints matching the frozen client enums; soft deletes via deleted_at where noted; created_at/updated_at on all mutable tables.

14.2 Entity–Relationship Overview

```
users 1■N vehicles · users 1■N addresses · users 1■1 wallet
users 1■N wallet_transactions · users 1■N reward_ledger
users 1■1 notification_settings · users 1■N conversations 1■N chat_messages
users 1■N diagnoses
categories 1■N products 1■N product_specifications / product_reviews
mechanics 1■N mechanic_reviews · mechanics M■N mechanic_services
orders 1■N order_items · fuel_orders 1■1 price_estimates · fuel_orders 1■N
tracking_events · fuel_orders 1■1 invoices · mechanic_bookings 1■N
booking_events · mechanic_bookings 1■1 ratings
orders 1■N order_entries (unified Orders-tab view)
```

14.3 Core Tables

- **users** — profile, tier (free/pro), emergency contact; is_logged_in and theme_mode stay on-device (SharedPreferences), not in the DB.
- **vehicles / addresses** — user-owned, default-first ordering, one default per user.
- **wallet / wallet_transactions / reward_ledger** — balance, debit/credit ledger, signed reward points.

- **notification_settings** — 1-1 JSON round-trip with the client model.
- ****categories / brands / products / product_specifications / product_reviews / offers / coupons**** — the frozen marketplace catalog.
- **orders / order_items / order_entries** — parent orders, line items, and the unified Orders-tab feed (`order_entries` maps to the client `ordersList`; Marketplace inserts write here; Orders tab + Profile history read it).
- ****mechanics / mechanic_services / mechanic_categories / mechanic_bookings / booking_events / ratings**** — booking state machine with JSONB tracking snapshots.
- ****fuel_orders / price_estimates / fuel_stations / fuel_partners / tracking_events / invoices**** — fuel lifecycle with partner assignment and invoice generation.
- **conversations / chat_messages / diagnoses** — AI module persistence.

14.4 Indexes and Constraints

- Unique: `users.email`, `users.phone`, `coupons.code`, `orders.external_id` (MKP-*), `invoices.order_id`, `fuel_orders.id`, `order_entries.id`.
- Foreign keys with the frozen client IDs mapped to UUID PKs where the client uses prefixed keys (`veh-*`, `addr-*`, `txn-*`, `rew-*`, `m*`, `svc_*`, `station_*`, `partner_*`, `ai-*`, `diag-*`).
- CHECK constraints on status/enum columns; partial unique index for one default vehicle/address per user; `sort_order` indexes on category/offer/ product surfaces.

14.5 Future Schema

Additive Sprint 2+ evolution: JWT refresh tokens, server-validated coupon usage ledger, push-notification tokens, order quotes, subscription plans, corporate fleet accounts, and analytics event tables — all backward-compatible with the frozen entity shapes.

Chapter 15 — API Contract

15.1 General Conventions

Convention	Value
Base path	/api/v1 (Sprint 2 target)
Latency	simulated: AI 900ms, Profile 800ms, Home 800ms, Marketplace 700ms, Fuel 700ms
Failure injection	failForFirstCalls → typed network exceptions (e.g. AiNetworkException)
Errors	typed exceptions with user-facing message
Money	double (■ INR)
Timestamps	ISO-8601 strings on payloads
Versioning	additive field changes only; enum changes require a client-matching release

Frozen ID schemes: MKP-<year>-<0000>, ORD-*, FUEL-<year>-<0000>, INV-<orderId>, p-*, coupon-*, offer-*, rv-*/r*, station_*, partner_*, svc_*, m*, ai-*, m-*, diag-*, veh-*, (from 200), addr-*, (from 200), txn-*, rew-*, pay-*.

15.2 Endpoints (module → repository methods)

Module	Methods
Home	fetchHomeData() → HomeData
Auth	AuthRepository (local state; JWT in Sprint 2)
AI	fetchConversations(), sendMessage(id, msg), diagnoseVehicle({type, problem, symptoms})
Marketplace	fetchProducts/Categories/Brands/Offers, getCoupons(), createOrder({items, address, payment})
Mechanic	list/query mechanics, details + reviews, book service, summary → confirmation, track, submit rating, history
Fuel	getFuelTypes, getSavedVehicles, getFuelStations({lat, lng}), createOrder, acceptOrder, advanceStatus, cancelOrder, completeOrder, generateInvoice, getTracking, getOrderHistory
Profile	fetchProfile/saveProfile, vehicle CRUD + default, address CRUD + default, fetchWallet, fetchRewards, fetchStats, fetchOrders, notification settings get/save

15.3 Request / Response Example — AI Diagnosis

Request: POST /api/v1/ai/diagnose

```
{ "vehicle_type": "bike", "problem": "won't start", "symptoms": ["clicking"] }
```

Response:

```
{
  "id": "diag-1",
  "vehicle_name": "Honda Activa 6G",
  "vehicle_type": "bike",
  "problem": "won't start",
  "symptoms": ["clicking"],
  "possible_causes": ["Flat or weak battery", "Faulty starter relay"],
  "severity": "high",
  "estimated_cost": 1200,
  "recommended_action": "Try a jump start once...",
  "should_drive": false,
  "recommended_service": "Battery & Starting System Service",
  "confidence": 86,
  "timestamp": "2026-08-02T..."
}
```

15.4 Validation and Errors

Forms validate client-side (email format, password ≥ 6 , confirm match, vehicle/address fields, fuel quantity clamps). Server (Sprint 2) must validate all inputs and return the typed error format so loading/empty/error/retry UI remains exercisable.

15.5 Authentication

No real auth at RC1 — login state is on-device. Sprint 2 adds Firebase Auth + JWT; the repository seam absorbs the change with no UI modification.

Chapter 16 — Testing

16.1 Strategy

Four-layer pyramid: unit (models, services, utilities) → widget (screens, widgets, forms) → integration (critical flows against the production provider graph) → manual QA (smoke, regression, device matrix, performance). Regression is fed by the integration suites.

16.2 Inventory (162 tests)

File	Count	Coverage
test/ai_module_test.dart	25	AI home, chat, diagnosis, history, pin/refresh merge, retry/failure
test/fuel_module_test.dart	37	stations, booking, price, lifecycle, tracking, invoice, history, failure
test/marketplace_module_test.dart	43	catalog, categories, cart, checkout, orders, wishlist, coupons, responsive
test/profile_module_test.dart	30	profile, vehicles, addresses, wallet, rewards, settings, order feed
test/mechanic_module_test.dart	10	mechanic list, details, booking, AI vehicle form
test/vehicle_location_test.dart	8	vehicle location flow
test/home_dashboard_test.dart	3	dashboard render + sections
test/integration/runtime_marketplace_flow_test.dart	2	end-to-end flow on the production graph
test/widget_test.dart	4	rating semantics, touch targets + tooltips, quick-services grid
Total	162	

16.3 Gates

```
flutter analyze    # MUST report: No issues found!
flutter test      # MUST report: All tests passed! (162)
```

- Module tests wrap real repositories/providers — no test-local graphs for integration assertions.
- The runtime flow test boots `buildRootProviders()` + `MyApp`, asserting a single provider identity and a single `Navigator` across the whole flow.
- Responsive tests render at 320/360/390/412/600/768dp in light + dark and assert zero overflow.
- A11y tests assert merged semantics and touch-target sizes.

- Failure/retry paths exercised deterministically via `failForFirstCalls`.

16.4 Runtime QA, Accessibility, Performance

- Runtime QA: loading/empty/error states per module; no real HTTP in mock paths; state retention across the 5-tab `IndexedStack`.
- Accessibility: semantics, tooltips, touch targets, reduced motion.
- Performance: no rebuild-every-tick listeners; timers isolated in self-contained widgets; `context.select` narrow rebuilds; controllers disposed.

Chapter 17 — Sprint History

Sprint	Deliverable	Outcome
0.0.1 (2026-07-20)	Project initialization, folder structure, pubspec	■
1.1–1.3 (0.1.0)	Splash, onboarding, authentication, basic navigation	■
1.4 (0.2.0)	Material 3 theme, widget library (14), home dashboard, bottom nav, AI chat	■
1.5 (0.3.0)	Dark mode, premium splash/home/login/onboarding/drawer, touch targets	■
1.6 (0.4.0)	Mechanic booking module (complete 9-step flow)	■
1.6.1–1.6.4 (0.4.0+/0.5.x/0.6.0)	Responsive system, overflows, mechanic card polish, brand selector	■
D1 (1.0.0)	Documentation blueprint set (7 docs), Mermaid diagrams, doc standards	■
D5.1 (1.1.0)	Repository hygiene, file renames, asset cleanup	■
1.7A (1.2.0)	Fuel delivery foundation, GPS fallback	■
1.8	Marketplace catalog/cart/checkout/wishlist + Orders integration	■
1.9	AI Assistant (chat, diagnosis, history)	■ 25 tests
1.9A	Profile module (30/30 tests)	■
1.9B (1.9.0)	RC1 certification: 3 P0 fixes, 29 dead files removed, 9 cert docs	■ 159/159
1.9b close (1.9.1)	Navigation, perf, a11y, responsive, code-quality polish	■ 162/162
1.9b final review (1.9.2)	Four-part audit, reduced-motion + token fixes, docs	■ 162/162

17.1 Lessons Learned

- Emergency apps need first-class failure paths — mock latency + failure injection made retry UI real instead of aspirational.
- A single provider graph (built in one place) prevents cross-tab and refresh inconsistencies (the AI triple-repository bug was a direct lesson).
- IndexedStack keeps tab state but requires explicit cross-tab notification (the OrderStore).
- Dead code referencing an absent backend silently regresses — remove it at freeze time and keep analyze at 0 issues.
- Design tokens and context getters make dark mode structural rather than per-widget branching.

17.2 Architecture Evolution

v1 prototype (legacy `lib/Auth`, `lib/home`, `lib/mechanic`, per-screen mock data, legacy HTTP seams) evolved into the v2 feature-first architecture (`lib/features/*` with models → repositories → providers → screens → widgets → services), a frozen design system, and a single `app_wiring.dart` provider graph. Legacy documents were archived, not deleted.

Chapter 18 — Deployment Roadmap

18.1 Sprint 2 (Backend Integration)

- Implement the frozen API contract with **FastAPI**; PostgreSQL migration per the database blueprint; Redis cache; Firebase Auth + JWT; Gemini-based AI diagnosis (RAG over the FAISS knowledge base); OpenStreetMap tiles; deploy backend, migrate, and wire repositories to the HTTP client — with zero UI changes and the full 162-test suite as the regression gate.

18.2 Sprint 3 (Production Polish)

Performance optimization, error handling and edge cases, accessibility audit, device-matrix QA, store preparation (Play Store listing, privacy policy), version bump, and release checklist completion (APK/app-bundle build, physical device smoke test).

18.3 CI/CD

GitHub Actions pipeline: `push` → `dart analyze` → `flutter test` → `build APK` → `build app bundle` → `artifact`. Distribution: `internal testing` → `closed alpha` → `open beta` → `production`. Version strategy: `dev-{hash}` / `alpha-{build}` / `beta-{build}` / `rc-{build}` / `{major}.{minor}.{patch}`.

18.4 Monitoring, Logging and Security

Crashlytics (crash-free > 99.5%) and Firebase Analytics (DAU, conversion, retention) planned; LOG_LEVEL env for logging. Security: no secrets in code (.env git-ignored), input validation everywhere, Firebase Security Rules, no PII stored on-device, JWT in Sprint 2. Android: minSdk 21, targetSdk 34.

Chapter 19 — Future Scope

Scope	Description
Admin dashboard (Sprint 5)	Web portal: user management, analytics and reports, AI monitoring
Partner app (Sprint 4)	Mechanic/vendor app: job acceptance, navigation, earnings dashboard
Fuel + parts partner portals	Station and seller onboarding, listings, earnings
Analytics	Business and fleet analytics dashboards
Subscription	Plans for frequent users; corporate fleet accounts
Enterprise	B2B AI diagnosis API (■999/fleet/month)
Product future	Voice-activated SOS, predictive maintenance alerts, insurance claim integration, multi-language (Hindi/Telugu/Tamil), offline cached mechanic data, deep links

Chapter 20 — Known Limitations and Risk Register

20.1 Known Limitations (accepted at RC1)

- All data is **mock/in-memory**; Sprint 2 replaces repository internals.
- Real-time tracking is **simulated** (TrackingInfo from in-memory state).
- Coupons are **not server-validated**.
- No real **auth backend** (login state persisted on-device only).
- Home teaser cards (marketplace / nearby / activity) are ****intentional static placeholders**** with no entity IDs; wiring is Sprint 2 scope.
- Roadmap features intentionally show "coming soon": Battery, Towing, full activity history, nearby service details.
- **Accepted contrast limits:** white on brandOrange ≈ 3.37:1 and darkPrimary ≈ 2.86:1 fall below WCAG AA for body text; kept as brand-mandated, revisit at the Sprint 2 design sign-off.
- **P3 visual debt:** a few legacy screens carry hardcoded hex colors or

off-scale radii (main.dart, auth_scaffold.dart, starting_screen/, emergency_card.dart, mechanic_home_screen.dart).

- **Rating-shorthand guard:** review.author.substring(0, 1) is safe with mock data; guard required once real backend data arrives.
- **P3 defensive:** loadReviews / cancelActiveBooking / completeActiveBooking lack try/catch; plaintext remember_me_password in SharedPreferences (mock-acceptable).

20.2 Risk Register

Risk	Severity	Mitigation
Sprint 2 contract drift	Medium	Frozen API contract + repository seams
Simulated tracking ≠ real geo	Low	Tracking payload shape frozen
Debug flags leaking into prod	Low	Runtime trace + dev flags removed
On-device login state only	Low	Sprint 2 real auth
Brand contrast < WCAG AA on orange	Low (accepted)	UI body text uses darker tokens
Home teaser cards static	Low (accepted)	Intentional placeholders
GPS denied/disabled	High	State machine + manual address fallback
Network failure during booking	High	Offline queue, retry logic, cached data
Mechanic no-show/cancel	High	Penalty system, auto-reassign, notifications
AI misdiagnosis	Medium	Confidence threshold < 60% shows disclaimer
Data privacy breach	Medium	Firebase rules, no on-device PII, encryption
Low-end device crashes	Medium	Responsive scaling, Crashlytics
Payment failure	High	Idempotency keys, exponential backoff

Chapter 21 — Appendix

21.1 Glossary

Term	Meaning
RC1	First release candidate
Frontend Lock Candidate	The frozen, audited frontend awaiting release approval
Repository seam	The interface that isolates UI from data source
Mock repository	In-memory data source simulating latency and failure
IndexedStack	Shell keeping all 5 tabs mounted and stateful
orderStore / ordersList	Shared singletons for the unified order feed
RAG	Retrieval-augmented generation (planned AI architecture)
NFR	Non-functional requirement

21.2 Packages

flutter_map, latlong2, geolocator, permission_handler, flutter_map_tile_caching, flutter_map_cancellable_tile_provider,/cupertino_icons, google_nav_bar, device_preview, flutter_dotenv, http, provider, nested, shared_preferences, flutter_lints (dev), flutter_test (dev). See pubspec.yaml.

21.3 Licenses

All packages are used under their respective open-source licenses (predominantly BSD-3-Clause and MIT). This document is proprietary and confidential. Full third-party license texts are recorded in the project's dependency license notes.

21.4 Credits

Mecha Connect Engineering Team; contributors to the Flutter/Dart ecosystem and all open-source packages used; Google (Flutter, Gemini); OpenStreetMap contributors.

21.5 References

- FRONTEND_LOCK_REPORT.md — freeze governance
- QA_CERTIFICATION_REPORT.md — certification evidence
- PROJECT_STATUS_REPORT.md — status, next steps, risks
- FRONTEND_ARCHITECTURE.md — provider graph, module tree, seams
- UI_DESIGN_SYSTEM.md — frozen design tokens

- NAVIGATION_MAP.md — full navigation maps
- API_CONTRACT.md — Sprint 2 API contract
- DATABASE_BLUEPRINT.md — Sprint 2 schema
- docs/01_product/* — PRD, business model, feature specs, roadmap, risk analysis (planning source material)
- docs/03_development/* — changelog, contributing, test plan, installation, deployment
- docs/05_reports/* — sprint reports
- docs/archive/* — superseded historical documents

21.6 Version History

Version	Date	Change
1.0.0	2026-08-05	RC1 release — rewritten as the official 21-chapter Master Handbook (supersedes v2.0.0 handbook draft)
2.0.0	2026-08-05	Handbook draft (Sprint 1.9b) — replaced by this release
1.0	—	Original engineering handbook (archived as docs/archive/MASTER_ENGINEERING_HANDBOOK_v1.0.md)

End of handbook. This document is the official technical reference for the Mecha Connect RC1 release and the Sprint 2 backend integration.